Репозиторий
github link

Идея

В этой работе мы проверяем, может ли добавка по оценке больной авторегрессионной vision-language модели (VLM) улучшать семантическое соответствие изображения текстовому запросу в text-to-image генерации диффузионной модели. Мотивация в том, что современные VLM уже содержат достаточные знания о композиции объектов, отношениях, количествах и редких семантических сочетаниях, а потому их сигнал потенциально можно использовать не только для post оценки, но и непосредственно для управления траекторией генерации.

Идея близка к классическому classifier guidance для диффузионных моделей, но вместо градиента классификатора используется градиент функции соответствия изображения и текста, вычисленный с помощью VLM. Важно, что речь идёт именно о больших авторегрессионных моделях. Так что метрику качества мы будем от них получать, как текстовый ответ на вопрос о соответствии изображения промту - Visual Question Answering (VQA). Далее мы будем называть наш алгоритм или VLM guidance или VQA score пайплайном.

Так же, кривогульной является мысль о том, что вернормированная семантика изображения (крупные формы, цвета, отношения объектов) задаётся на ранних итерациях денойзинга. Наш пайплайн нацелен на то, чтобы подтолкнуть генерацию ранних латентов в сторону большего семантического соответствия промту.

Введение в область и обозначения

В стандартной диффузионной генерации изображение получается итеративным денойзингом латентных переменных

$$z_t \to z_{t-1} \to \dots \to z_0$$

Шаг такого разложения $z_t \to z_{t-1}$ основывается на предсказании диффузионной модели $\epsilon_\theta(z_t, p, t)$,где z_t - латент, t - номер шага разложения и p - эмбеддинг промта.

В конце генерации, при получении z_0, происходит отображение z_0 в пространство изображений $I = \mathrm{VAE}(z_0)$

Так же стандартные алгоритмы позволяют по z_t и предсказанному $\epsilon_\theta(z_t)$ например строить оценку $\hat{z}_0^t$ на z_0

Алгоритм

Наш пайплайн строится вокруг $\mathcal{L}_{\text{vlm}}(I)$ - лосса посторенного на VLM оценке соответствие семантики изображения промту. Мы берем дифференциал этого лосса по латенту z_t($\nabla_{z_t}\mathcal{L}_{\text{vlm}}(I)$ и делаем M шагов градиентного спуска по z_t.

Чтобы избежать трудоемкого вычисления градиента сквозь все итерации денойзинга $\nabla_{\theta}\mathcal{L}_{\text{vlm}}(I)$ мы считаем градиент лосса по оценке на изображение: $\nabla_I\mathcal{L}_{\text{vlm}}(\hat{I}_t)$, где $\hat{I}_t = \mathrm{VAE}(\hat{z}_0^t)$

В нашем пайплайне лосс считается по формуле

$$\mathcal{L}_{\text{vlm}}(I) = \mathrm{CE}(\mathrm{VLM}(I, q)[:|Y|], Y)$$

где p - оригинальный промт генерации изображения $q(p)$ - функция оборачивающая промт генерации в вопрос о соответствии изображения промту, $\mathrm{VLM}(I, q)[:|Y|]$ - предсказанное VLM распределение вероятностей на первый токен ответа на вопрос о соответствии изображения промту генерации.

Полученный таким образом градиент нормировался и использовался в градиентном спуске. В общем случае за раз мы делали M шагов градиентного спуска:

$$z_t \leftarrow z_t - \lambda \frac{\nabla_{z_t}\mathcal{L}_{\text{vlm}}(\hat{I}_t)}{|\nabla_{z_t}\mathcal{L}_{\text{vlm}}(\hat{I}_t)|}$$

Такие гладкие градиентные спуски повторяется первые K итераций разсуждения, что позволяет приблизить семантики ранних латентов к верной семантике промта.

Целиком наш пайплайн можно описать следующим псевдокодом

Algorithm. VLM-guided denoising of early latents.
Input: prompt p, initial noise z_{T}, diffusion model ϵ_θ, VAE decoder, VLM, hyperparameters K, M, λ.
Output: generated image I.

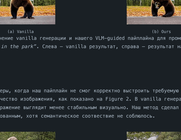
For $t = T, T-1, \dots, 1$:
  If $t > T - K$:
    For $j = 1, \dots, M$:
      Predict $\hat{\epsilon}_t$ from z_t using $\epsilon_{\theta,t,p,\varnothing}$.
      Decode current image estimate: $\hat{I}_t = \mathrm{VAE}(\hat{z}_0^t)$.
      Form question $q(p)$.
      Compute VLM loss $\mathcal{L}_{\text{vlm}} = \mathrm{CE}(\mathrm{VLM}(\hat{I}_t, q)[:|Y|], Y)$.
      Compute gradient $g_t = \nabla_{z_t}\mathcal{L}_{\text{vlm}}$.
      Normalize gradient $g_t \leftarrow \frac{g_t}{|g_t|_2}$.
      Update latent $z_t \leftarrow z_t - \lambda g_t$.
    Perform ordinary diffusion update $z_t \leftarrow \epsilon_\theta(z_t, p, t)$.
Decode final latent: $I = \mathrm{VAE}(z_0)$.
Return I

Экспериментальный сетап

В качестве диффузионной модели использовалась stable diffusion 1.5, $T = 30$ итераций денойзинга, $K = 2$.
В качестве VLM использовалась clip-flant5-xxl.
Наилучие результаты достигались при $K = 10$, $M = 3$, $\lambda = 0.02$

Качественные сравнения

Результаты получились неоднозначные. Есть примеры улучшения соответствия изображения промту, как на Figure 1, слева, vanilla пайплайн не справляется с формированием правильно семантики, тогда как справа наш метод делает генерацию ближе к промту.



(a) Vanilla (b) Ours

Figure 1: Сравнение vanilla генерации и нашего VLM-guided пайплайна для промта *"A bear does a handstand in the park"*. Слева – vanilla результат, справа – результат нашего метода.

Так же есть примеры, когда наш пайплайн не смог корректно выстроить требуемую семантику, но улучшил общее качество изображения, как показано на Figure 2. В vanilla генерации также нет кролика, а изображение выглядит менее стабильным визуально. Наш метод сделал изображение более структурированным, хотя семантическое соответствие не соблюлось.

(a) Vanilla (b) Ours

Figure 2: Сравнение vanilla генерации и нашего VLM-guided пайплайна для промта *"an elephant under a rabbit"*. Слева – vanilla результат, справа – результат нашего метода. Несмотря на то, что требуемая семантика *"Elephant under a rabbit"* не воспроизводится полностью, наш пайплайн дал более стабильную и визульно согласованную генерацию.

Но есть и примеры, когда наш пайплайн и не смог корректно выстроить требуемую семантику и ухудшил общее качество изображения, как показано на Figure 3. В vanilla генерации также две обезьяны, вместо обезьяны и собаки, но изображение выглядит более качественным. Наш метод сделал изображение визульно менее привлекательным и не исхаженным. Мы предполагаем что это из-за того, что stable diffusion не выиграла distribution shift, происходящий при наших итерациях градиента.

(a) Vanilla (b) Ours

Figure 3: Сравнение vanilla генерации и нашего VLM-guided пайплайна для промта *"a dog and a monkey"*. Слева – vanilla результат, справа – результат нашего метода. Требуемая семантика "a dog and a monkey" не воспроизводится ни одним из пайплайнов, но как несколько ухудшил визульное качество изображения.

Количественные сравнения

Мы провели сравнения по метрикам трех пайплайнов генерации. Vanilla – оригинальный пайплайн stable diffusion 1.5, VQA score – предложенный нами пайплайн и генерации Flux1-dev – значительно более сильной моделью чем stable diffusion 1.5

Количественные результаты также получились неоднозначными. Чтобы понять, как guidance влияет не только на отдельные примеры, но и на распределение качества по датасету, мы сравнили пайплайны по нескольким метрикам: Alignment Score, Quality Score и CLIP Score. Соответствующие распределения показаны на Figure 4, Figure 5 и Figure 6.

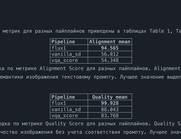


Figure 4: Распределение Alignment Score для разных пайплайнов. Alignment Score вычислялся как оценка соответствия семантики изображения исходному промту, полученная с помощью GPT-4o. Чем больше больших значений, тем лучше. Видно, что для нашего VQA Score пайплайна значения метрики в диапазоне 60-80 встречается реже, чем для vanilla пайплайна.

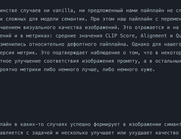


Figure 5: Распределение Quality Score для разных пайплайнов. Quality Score вычислялся как оценка визульного качества изображения, полученная с помощью GPT-4o без учета соответствия промту. Чем больше больших значений, тем лучше. Видно, что наш VQA Score пайплайн расширяет распределение: в части случаев качество заметно улучшается, в части – заметно ухудшается. В среднем результат оказывается немного хуже, чем у vanilla пайплайна.

Figure 6: Распределение CLIP Score для разных пайплайнов. CLIP Score вычислялся как cosine similarity между CLIP-эмбеддингами изображения и промта. Чем больше больших значений, тем лучше. Видно, что наш пайплайн расширяет распределение метрики в обе стороны по сравнению с vanilla пайплайном, при этом среднее значение остается примерно на том же уровне.

Средние значения метрик для разных пайплайнов приведены в таблицах Table 1, Table 2 и Table 3.

Pipeline	Alignment mean
flux1	94.565
sd15_vqa_sd	56.617
sd15_sd	54.348

Table 1: Сводка по метрике Alignment Score для разных пайплайнов. Alignment Score отражает соответствие семантики изображения тексторому промту. Лучшее значение выделено полужирным.

Pipeline	Quality mean
flux1	89.920
sd15_vqa_sd	60.491
sd15_sd	63.70

Table 2: Сводка по метрике Quality Score для разных пайплайнов. Quality Score отражает визульное качество изображения без учета соответствия промту. Лучшее значение выделено полужирным.

Pipeline	CLIP mean
flux1	**0.2994**
sd15_vqa_sd	0.2617
sd15_sd	0.2687

Table 3: Сводка по метрике CLIP Score для разных пайплайнов. CLIP Score вычислялся как cosine similarity между CLIP-эмбеддингами изображения и промта. Лучшее значение значение выделено полужирным.

В целом, в большинстве случаев он хуже, чем предложенный нами пайплайн с задачей генерации сложных для модели семантик. При этом наш пайплайн с переменным успехом справился с улучшением визульного качества изображений. Это отражается и на приведенных примерах изображений и в метриках: среднее значение CLIP Score, Alignment и Quality практически не изменилось относительно дефолтного пайплайна. Однако для нашего метода немного увеличилась дисперсия метрик. Это подтверждает наблюдение о том, что в некоторых сценариях наблюдается заметное улучшение соответствия изображения промту, а в остальных сценариях примерно равнозначно метрики либо немного лучше, либо немного хуже.

Вывод и планы

Я того, наш пайплайн с каких-то случаях успешно формирует в изображении семантики промта, в остальных не справляется с задачей и несколько улучшает или ухудшает качество изображения. Мы предполагаем, что такое поведение обусловливается недостаточной степенью влияния на код генерации и при этом пограничной величиной distribution shift для диффузионной модели, возникающим при нашем гайдансе.

Мы предполагаем, что если взять более робастную диффузионную модель, то она окажется менее чувствительной к distribution shift, что позволит, усилив степень гайдинга, получить более выразительные результаты.

Я планирую запустить этот пайплайн гайдинга на Flux1-dev. Ее метрикам она себя зарекомендовала как значительно более робастная чем stable diffusion 1.5.